MAGYARGRAVÍR WEBÁRUHÁZ

Projekt dokumentáció

2026.

Tartalomjegyzék

1. Felhasznált Technológiák 8
1.1. Frontend 8
1.1.1. HTML 8
1.1.2. CSS 9
1.1.3. JavaScript 10
1.2. Backend 10
1.2.1. C# (C-Sharp) 11
1.2.2. ASP.NET Core 11
1.2.3. Razor Pages 11
1.3. Adatbázis 11
1.3.1. Entity Framework Core 12
1.3.2. SQLite adatbázis 12
1.3.3. Adatbázis séma 13
2. A projekt szerkezete 14
2.1. Mappa struktúra 14
2.1.1. Models 14
2.1.2. Data 14
2.1.3. Migrations 15
2.1.4. Pages 15
2.1.5. wwwroot 16
2.1.6. Helpers 16
2.1.7. Connected Services 16
2.1.8. Dependencies 16
2.1.9. Properties 16
2.1.10. Program.cs 17
2.1.11. appsettings.json 17
2.1.12. .gitignore 17
2.1.13. app.db 17
2.1.14. MagyarGravir.Shop.http 17
2.1.15. _Layout.cshtml & _AdminLayout.cshtml 18

2.1.16. _ViewImports.cshtml 18
2.1.17. _ViewStart.cshtml 18
3. Felhasználói Dokumentáció 19
3.1. Webshop felület 19
3.1.1. Főoldal 19
3.1.2. Termékek megtekintése 20
3.1.3. Kosár 20
3.1.4. Bejelentkezés 21
3.1.5. Regisztráció 21
3.1.6. Lábléc 22
3.2. Admin felület 22
3.2.1. Termékkezelés 23
3.2.2. Kategóriakezelés 24
3.2.3. Rendelések kezelése 24
4. Tesztelés 26
5. Összefoglalás 27
6. Irodalomjegyzék 28

BEVEZETÉS

A projekt célja egy saját fejlesztésű reszponzív webáruház létrehozása volt, amely egyedi gravírozott termékek értékesítésére szolgál. A webshop lehetőséget biztosít arra, hogy a felhasználók böngésszenek a különböző termékek között, kosárba helyezzék azokat, illetve megrendelést adjanak le.

A webáruház külön adminisztrációs felületet is tartalmaz, amelyen keresztül kezelhetőek a termékek feltöltése, képek, árak és kategóriák, valamint a beérkezett rendelések. A rendszer célja egy könnyen kezelhető, egyszerű struktúrájú webshop megvalósítása modern webfejlesztési technológiák használatával.

A projekt fejlesztése során fontos szempont volt az átlátható kódszerkezet, a bővíthetőség, valamint a gyakorlati használhatóság egy modern dizájnos és könnyen áttekinthető megjelenéssel.

1. FELHASZNÁLT TECHNOLÓGIÁK

1.1. FRONTEND

A frontend a webshop felhasználói felületét biztosítja, amely közvetlen kapcsolatban áll a felhasználóval. Itt történik a weboldal kinézetének és interaktivitásának megjelenítése, amelyet *HTML*, *CSS* és *JavaScript* segítségével valósítottunk meg. Ez határozza meg az oldal megjelenését, struktúráját és interaktivitását.

A frontend kialakításánál fontos szempont volt a reszponzivitás, tehát az oldal különböző eszközökön (asztali gép, tablet, mobiltelefon) is megfelelően jelenjen meg. Emellett törekedtünk arra, hogy az oldal gyors betöltési idővel és gördülékeny működéssel rendelkezzen.

1.1.1. HTML

A *HTML* (HyperText Markup Language) a weboldalak szerkezetének leírására szolgáló jelölőnyelv. A webshop felületének alapját *HTML* struktúrák adják, amelyek meghatározzák az oldalak felépítését, például a menüket, űrlapokat, termékkártyákat és egyéb elemeket.

A projekt során szemantikus *HTML* elemek használatára törekedtünk (pl. <header>, , <img> stb.), amelyek nemcsak a kód olvashatóságát javítják, hanem a keresőoptimalizálás (SEO) szempontjából is előnyösek.

A *HTML* elemek segítségével kerültek kialakításra a termékkártyák, űrlapok (pl. bejelentkezés, regisztráció), valamint a navigációs menü.

1.1.2. CSS

A *CSS* (Cascading Style Sheets) felel a weboldal vizuális megjelenéséért. A projekt során a *CSS* segítségével kerültek kialakításra az oldalak színei, elrendezése, valamint a különböző felhasználói felületi elemek megjelenése. A projekt során külön stílusfájlok kerültek létrehozásra az egyes oldalakhoz, amely segíti a kód modularitását és átláthatóságát.

A webshop külön stíluslapok például:

- admin felület *(admin.css)*
- kosár oldal *(cart.css)*
- rendelés oldal *(checkout.css)*

A stílusok kialakításánál fontos szempont volt:

- az egységes arculat
- a jól olvasható tipográfia
- a megfelelő kontraszt
- a felhasználói élmény javítása (pl. hover effektek)
- modern dekoratív megjelenítés

Ezen kívül a layout kialakítása során modern *CSS* megoldásokat (pl. flexbox) alkalmaztunk az elemek elrendezésére.

```
1   @import url('https://fonts.googleapis.com/css2?family=Cinzel:wght@500;700&family=Poppins:wght@300;400;500;600&fa
2
3   * {
4       box-sizing: border-box;
5   }
6
7   body.mg-body {
8       margin: 0;
9       min-height: 100vh;
10      background: radial-gradient(circle at top, #020617 0, #000 55%, #020617 100%);
11      color: #e5e7eb;
12      font-family: 'Poppins', system-ui, -apple-system, BlinkMacSystemFont, sans-serif;
13  }
14
15  /* HEADER */
16
17  .mg-header {
18      position: sticky;
19      top: 0;
20      z-index: 30;
21      backdrop-filter: blur(18px);
22      background: linear-gradient(to bottom, rgba(15,23,42,0.96), rgba(15,23,42,0.6), transparent);
23      border-bottom: 1px solid rgba(148,163,184,0.4);
24  }
25
26  .mg-header-inner {
27      max-width: 1120px;
28      margin: 0 auto;
110 %   No issues found                                   Ln: 1, Ch: 1   SPC   CRLF   UTF-8
```

1. ábra: CSS

1.1.3. JavaScript

A *JavaScript* a kliensoldali interaktivitásért felel. Segítségével különböző dinamikus műveletek valósíthatók meg a felhasználói felületen. Segítségével a felhasználói interakciók azonnali visszajelzést adnak, így nem szükséges minden művelethez az oldal újratöltése.

A webshopban *JavaScriptet* használunk például:

- felhasználói interakciók kezelésére
- kosár működésének segítésére
- dinamikus felületfrissítésre

Ez jelentősen javítja a felhasználói élményt és gyorsabbá teszi az alkalmazás használatát.

1.2. BACKEND

A backend a szerveroldali működést biztosítja, és kezeli a webshop logikáját, gyakorlatilag a webáruház „motorja”. Itt történik az üzleti logika, a felhasználói műveletek feldolgozása, az adatbázis-műveletek és a rendelések kezelése. Az általunk tervezett webshop szerveroldali logikája *C# (C-Sharp)* nyelven íródott, *ASP.NET Core* keretrendszert és a *Razor Pages* architektúrát használja, amely lehetővé teszi a weboldalak és a hozzájuk tartozó logika együttes kezelését.

A backend feladata többek között:

- adatok validálása
- jogosultságkezelés
- rendelési folyamat kezelése
- adatbázis műveletek végrehajtása

A backend stabil működése kulcsfontosságú a rendszer megbízhatósága szempontjából.

1.2.1. C# (C-Sharp)

A *C#* egy objektumorientált programozási nyelv, amelyet a *Microsoft* fejlesztett. A *.NET* keretrendszerhez tartozik, és lehetővé teszi a hatékony szerveroldali alkalmazásfejlesztést, valamint a backend logika, az adatkezelés és a webszolgáltatások egyszerű megvalósítását.

1.2.2. ASP.NET Core

Az *ASP.NET Core* egy modern, nyílt forráskódú webfejlesztési keretrendszer, amelyet a *Microsoft* fejlesztett.

1.2.3. Razor Pages

A *Razor Pages* az *ASP.NET Core* keretrendszer része, amely egy oldal-alapú architektúrát biztosít webalkalmazásokhoz. Lehetővé teszi, hogy minden weboldalhoz egy *Page Model* tartozzon, ahol a szerveroldali logika található, így a *HTML* és a hozzá kapcsolódó *C#* kód együtt, áttekinthetően kezelhető. A *Razor Pages* a *Razor* jelölőnyelvet használja a dinamikus *HTML*-generáláshoz, és támogatja az újrahasznosítható komponensek, *layout-ok* és *Tag Helper-ek* alkalmazását.

A *Razor Pages* előnyei:

- jól struktúrált kód
- könnyű routing kezelés
- szerveroldali renderelés
- beépített biztonsági funkciók

1.3. ADATBÁZIS

Az adatbázis tárolja a webshop adatait, például a termékeket, felhasználókat és rendelések információit. A projekt *Entity Framework Core* segítségével kommunikál az adatbázissal, amely *SQLite*-ban van tárolva. Az adatbázis sémája meghatározza a táblák struktúráját és a köztük lévő kapcsolatokat, biztosítva a következetes adatkezelést és lekérdezéseket.

Az adatbázis használata lehetővé teszi:

- gyors adatlekérdezést
- adatok módosítását

- konzisztens adattárolást
- átlátható kapcsolatokat

1.3.1. Entity Framework Core

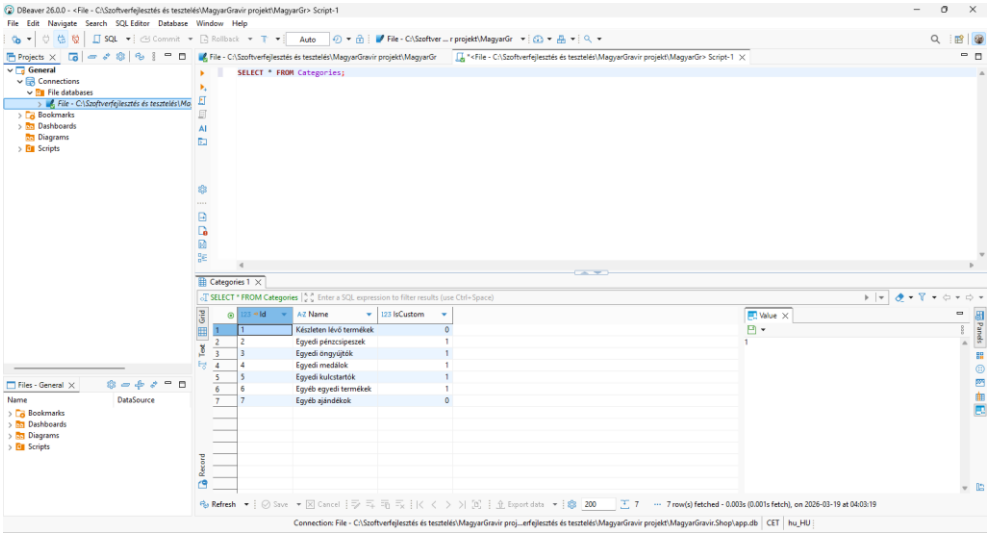
Az *Entity Framework Core* egy objektum-relációs leképező (*ORM*) keretrendszer, amely lehetővé teszi az adatbázissal való kommunikációt objektumokon keresztül. A webshopban az *Entity Framework Core* kezeli a termékek, kategóriák, felhasználók, rendelések, valamint a rendelési tételek adatbázis-műveleteit.

1.3.2. SQLite adatbázis

A projekt jelenlegi verziója *SQLite* adatbázist használ. Ez egy könnyű, fájl alapú adatbázis rendszer, amely külön szerver telepítése nélkül működik. A strukturált lekérdezési nyelvet (*SQL*) az adatok lekérdezéséhez, módosításához és kezeléséhez használják. Az *SQLite* platformfüggetlen, és *Windows*, *Linux*, *macOS*, mobil operációs rendszereken, például Androidon és *iOS*-en is futtatható.

Az adatbázis tárolja többek között a termékek adatait, a kategóriákat, a felhasználókat és a rendeléseket.

Az *app.db* adatbázis vizsgálatához a *DBeaver* programot használtuk. Az alábbi ábra egy SELECT * FROM Categories; lekérdezést és annak eredményét szemlélteti.

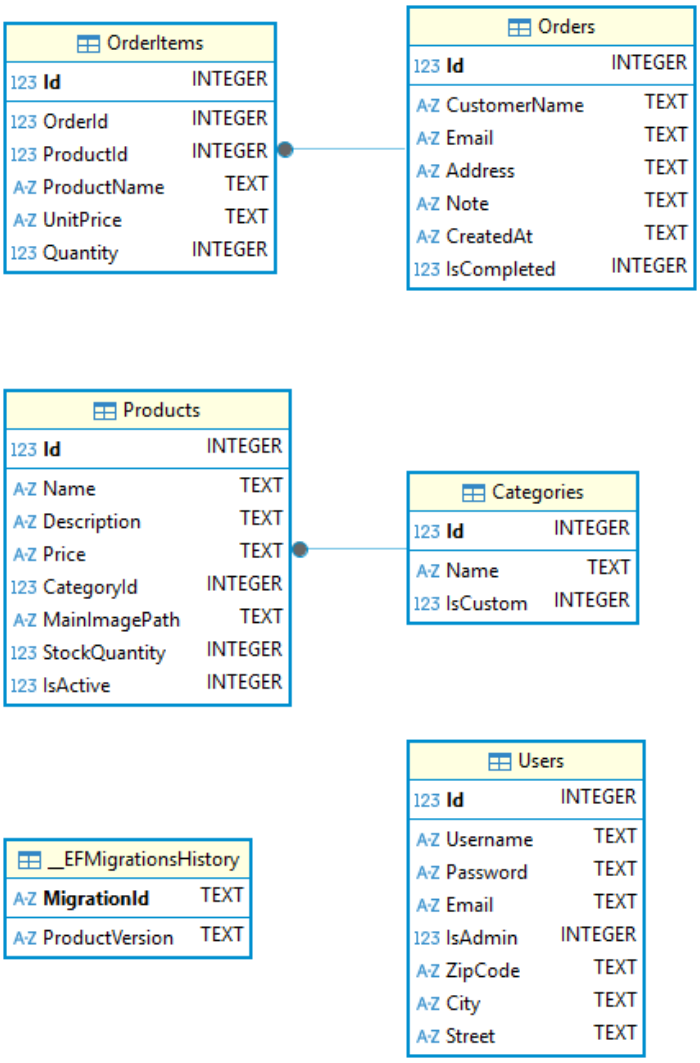


2. ábra: SQL lekérdezés DBeaver programban

1.3.3. Adatbázis séma

Az adatbázis séma az adatbázis felépítését írja le, vagyis megmutatja a táblákat, azok mezőit és a közöttük lévő kapcsolatokat. Segítségével könnyen áttekinthető az adatok struktúrája, ami megkönnyíti a lekérdezések készítését és a rendszer működésének megértését.

Az alábbi ábra az adatbázis sémáját mutatja ER diagram formájában, amely áttekinthetően szemlélteti a táblák közötti kapcsolatokat és a struktúrát.



3. ábra: Adatbázis séma

2. A PROJEKT SZERKEZETE

A projekt szerkezete a webshop különböző funkcionális részeinek logikai elkülönítésére épül. A mappák és fájlok kialakítása azt a célt szolgálja, hogy az alkalmazás felépítése átlátható, jól strukturált és könnyen karbantartható legyen.

A struktúra egyértelműen elválasztja egymástól a felhasználói felületet, az adatkezelést, a statikus erőforrásokat és a konfigurációs elemeket. Ennek köszönhetően a fejlesztés során az egyes komponensek egymástól függetlenül módosíthatók és bővíthetők.

A következőkben a projekt főbb mappái és azok szerepe kerül bemutatásra.

2.1. MAPPA STRUKTÚRA

2.1.1. Models

A *Models* mappa az alkalmazás adatmodelljeit tartalmazza, amelyek meghatározzák a webshopban kezelt adatok szerkezetét és az adatbázis tábláit.

Ezek a következők:

- Cart
- CartItem
- Category
- CreateModel
- DeleteModel
- EditModel
- Order
- Order
- OrderItem
- Product
- User

2.1.2. Data

A *Data* mappa az alkalmazás adatbázis-kezeléséhez kapcsolódó osztályokat tartalmazza. Ebben a mappában található az *AppDbContext.cs* fájl, amely az adatbázis és az alkalmazás közötti kapcsolatot biztosítja.

2.1.3. Migrations

Az adatbázis migrációk helye.
Feladata:

- adatbázis séma létrehozása
- módosítások verziózása

2.1.4. Pages

A *Pages* mappa tartalmazza a *Razor Pages* nézeteket és az azokhoz tartozó *PageModel* osztályokat. Ez a mappa az alkalmazás felhasználói felületének központi eleme, ahol az oldalak logikája és a *HTML* nézetek egy helyen találhatók.

Pages mappa főbb almappái:

- Shop (felhasználói felület)
- Admin (adminisztrációs felület)
- Account (bejelentkezés, regisztráció)

Minden Razor Page két részből áll:

- *.cshtml* (nézet)
  HTML + Razor syntax, amely megjeleníti az oldalt.
- *.cshtml.cs* (logika)
  a PageModel osztály, amely tartalmazza az oldal logikáját.

A *Pages* mappa tartalmaz néhány speciális fájlt, amelyek nem klasszikus *Razor Page-ek*, mert nem rendelkeznek *.cshtml.cs* kódfájllal.

Ezek a következők:

- _Layout.cshtml
- _ViewStart.cshtml
- _ViewImports.cshtml

2.1.5. wwwroot

A *wwwroot* a statikus fájlok tárolására szolgáló mappa.

Ide tartoznak:

- képek
- CSS fájlok
- JavaScript fájlok
- feltöltött termékképek

2.1.6. Helpers

Segédosztályok és kiegészítő funkciók.

Például:

- kosárkezelés
- formázások
- egyéb újrafelhasználható logika

2.1.7. Connected Services

Külső szolgáltatások és adatbázis kapcsolatok.

- SQL Server Database
- SQLite (Local)

2.1.8. Dependencies

A projekt által használt külső könyvtárak (NuGet csomagok).

2.1.9. Properties

Projekt konfigurációs fájlok.

2.1.10. Program.cs

Az alkalmazás belépési pontja. Ebben a fájlban történik a webalkalmazás konfigurálása és indítása.

2.1.11. appsettings.json

Az *appsettings.json* fájl az alkalmazás konfigurációs beállításait tartalmazza.

2.1.12. .gitignore

Meghatározza, mely fájlok és mappák ne kerüljenek verziókezelésbe. Ennek célja, hogy a Git csak a projekt forráskódját és releváns fájljait kövesse, míg a generált, ideiglenes vagy helyi beállításokat figyelmen kívül hagyja.

.gitignore fájl tartalma:

- .vs/
- bin/
- obj/
- app.db, app.db-shm, app.db-wal
- *.user
- *.suo

2.1.13. app.db

Az *app.db* fájl a projekt SQLite adatbázisát tartalmazza.

Kiegészítő fájlok:

- app.db-shm
- app.db-wal

(Ezek az SQLite működéséhez szükségesek.)

2.1.14. MagyarGravir.Shop.http

HTTP kérések tesztelésére szolgáló fájl (pl. API hívásokhoz).

2.1.15. _Layout.cshtml & _AdminLayout.cshtml

A *_Layout.cshtml* és az *_AdminLayout.cshtml* fájl a webalkalmazás közös sablonja, amely meghatározza az oldalak alapvető szerkezetét és megjelenését.

Ez a fájl tartalmazza azokat az elemeket, amelyek minden oldalon azonosak, például:

- fejléc
- navigációs menü
- lábléc
- stíluslapok *(site.css)*

A layout biztosítja, hogy a webalkalmazásunk minden oldala egységes kinézettel rendelkezzen. Az egyes oldalak tartalma a layouton belül, a *@RenderBody()* metódus helyén jelenik meg.

2.1.16. _ViewImports.cshtml

A *_ViewImports.cshtml* fájl a *Razor Pages* nézetekhez tartozó közös beállításokat és utasításokat tartalmazza. Feladata, hogy egységesen biztosítsa a nézetek számára a szükséges névtereket és segédeszközöket. A fájl használatával elkerülhető, hogy ezeket az utasításokat minden egyes oldalon külön meg kelljen adni, így a kód átláthatóbb és könnyebben karbantartható marad.

2.1.17. _ViewStart.cshtml

A *_ViewStart.cshtml* fájl a *Razor Pages* nézetek közös beállításait tartalmazza, és minden oldal renderelése előtt automatikusan lefut. Legfontosabb szerepe az alkalmazás egységes megjelenésének biztosítása, mivel itt adható meg az alapértelmezett layout oldal.

Ennek köszönhetően nem szükséges minden egyes *.cshtml* fájlban külön beállítani a layoutot, így csökken a kódismétlés és egyszerűbbé válik a karbantartás.

A *_ViewStart.cshtml*, a *_ViewImports.cshtml*, valamint a *_Layout.cshtml* fájlok nem tartoznak a „klasszikus“ Razor Pages oldalak közé, mert nem tartalmaznak önálló oldal logikát, csak közös beállításokat. Ezek a speciális fájlok nem rendelkeznek *.cshtml.cs* kódfájllal.

3. FELHASZNÁLÓI DOKUMENTÁCIÓ

3.1. WEBSHOP FELÜLET

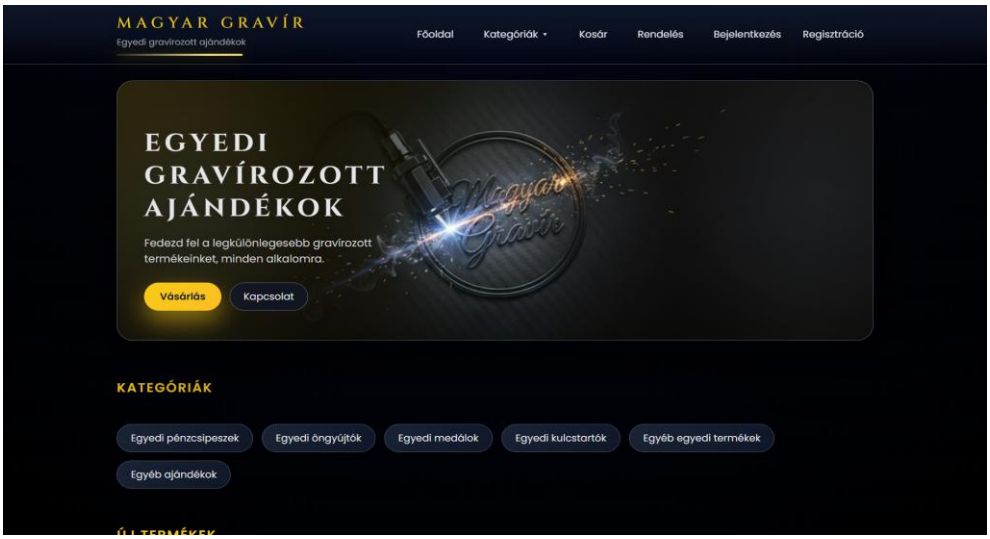
3.1.1. Főoldal

A főoldalon a webshop bemutatása található, valamint innen érhetők el a fő menüpontok.

A webshop elkészítésénél törekedtünk a dekoratív és modern megjelenítés mellett az egyszerű és intuitív kialakításra. A felhasználók így könnyen navigálhatnak az oldalon, és gyorsan megtalálhatják a keresett termékeket.

A navigációs menü segítségével a felhasználó elérheti:

- a termékeket
- a kosarat
- a bejelentkezési felületet
- a regisztrációs oldalt



4. ábra: Webshop kezdőoldal

3.1.2. Termékek megtekintése

A felhasználó a webshop termékeit kategóriák szerint böngészheti, az egyszerű és gyors navigáció és a könnyű keresés érdekében.

A termékek kártyaszerű formában jelennek meg, amelyek tartalmazzák:

- a termék nevét
- a termék képét
- az árát
- egy gombot a kosárba helyezéshez

3.1.3. Kosár

A kosárban jelennek meg azok a termékek, amelyeket a felhasználó meg kíván vásárolni. A kosár funkció lehetővé teszi, hogy a felhasználók több terméket gyűjtsenek össze vásárlás előtt.

A kosár oldalon lehetőség van:

- termék eltávolítására
- a rendelés leadására
- a mennyiségek módosítására
- a vásárlás folytatására

A rendelés során a rendszer eltárolja a rendelési adatokat az adatbázisban.

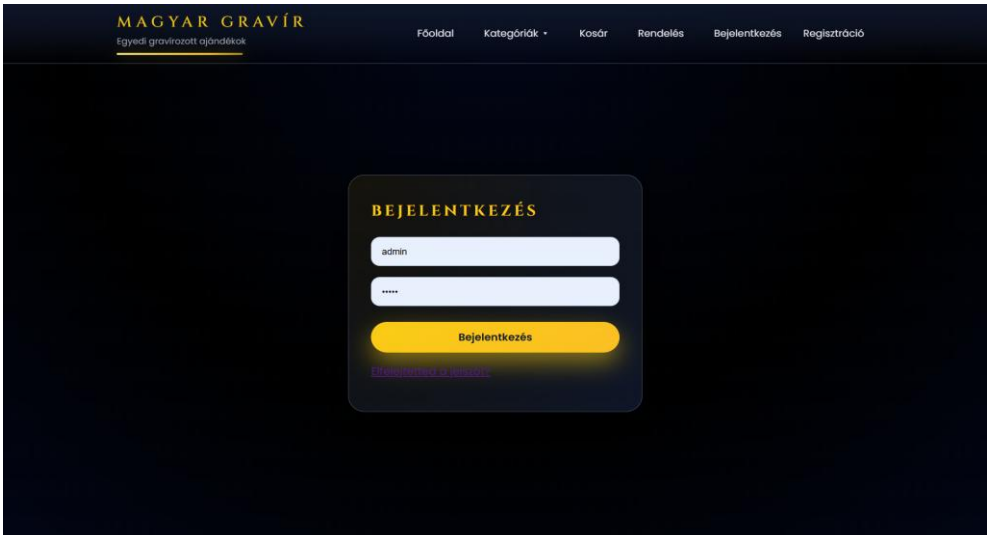
5. ábra: Kosár

3.1.4. Bejelentkezés

A bejelentkezési oldalon a regisztrált felhasználók a helyes adataik megadásával tudnak belépni. A bejelentkezéshez szükséges adatok és a jelszó elfelejtése funkció:

- felhasználónév
- jelszó (elfelejtett jelszó)

Sikeres bejelentkezés után a felhasználó hozzáfér a kosár és rendelési funkciókhoz.



6. ábra: Felhasználói Bejelentkezés felület

3.1.5. Regisztráció

A regisztráció során a felhasználó új fiókot hozhat létre. A regisztrációs űrlap kitöltése után az adatok az adatbázisba kerülnek. A regisztráció során szükséges megadniuk egy felhasználó nevet, célszerűen a saját nevük, valamint e-mail címet és jelszót, amelyet meg kell erősíteni az esetleges elgépelés lehetőségének elkerülése érdekében. Továbbá szükséges megadni a lakhelyükhöz tartozó, irányító számot, települést, valamint utca és házszámot.

Ezek az adatok a leadott rendelésnél felhasználhatók szállítási adatokként.

7. ábra: Regisztráció felülete

3.1.6. Lábléc

A webshop lábléce minden oldalon megjelenik, és a következő linkeket tartalmazza, amelyek mindegyike külön oldalon érhető el:

- ÁSZF - a felhasználási feltételek és jogi szabályozások elérhetősége.
- Kapcsolat - elérhetőséggel kapcsolatos információk.

Ezen kívül a lábléc tartalmazza a © (copyright) jelet és a „Minden jog fenntartva” feliratot, amely felhívja a felhasználó figyelmét arra, hogy a webshop tartalma jogi védelem alatt áll, és annak jogosulatlan felhasználása tilos.

A lábléc minden oldalon egységesen jelenik meg, biztosítva a fontos információk könnyű hozzáférhetőségét.

3.2. ADMIN FELÜLET

A webshop adminisztrációs felületet is tartalmaz, amely külön URL-en érhető el. Ez egy zárt rendszer, amelyhez csak jogosultsággal rendelkező felhasználók férhetnek hozzá. Az admin felületre történő belépés adminisztrátori felhasználónév és jelszó megadásával lehetséges, amely biztosítja, hogy illetéktelenek ne férhessenek hozzá az adatokhoz.

Az admin felület célja, hogy a webshop üzemeltetése egyszerűen és hatékonyan elvégezhető legyen egy központi felületen keresztül. Itt kezelhető a rendszer szinte minden fontos eleme, így nincs szükség közvetlen adatbázis-módosításra vagy fejlesztői beavatkozásra a mindennapi működés során.

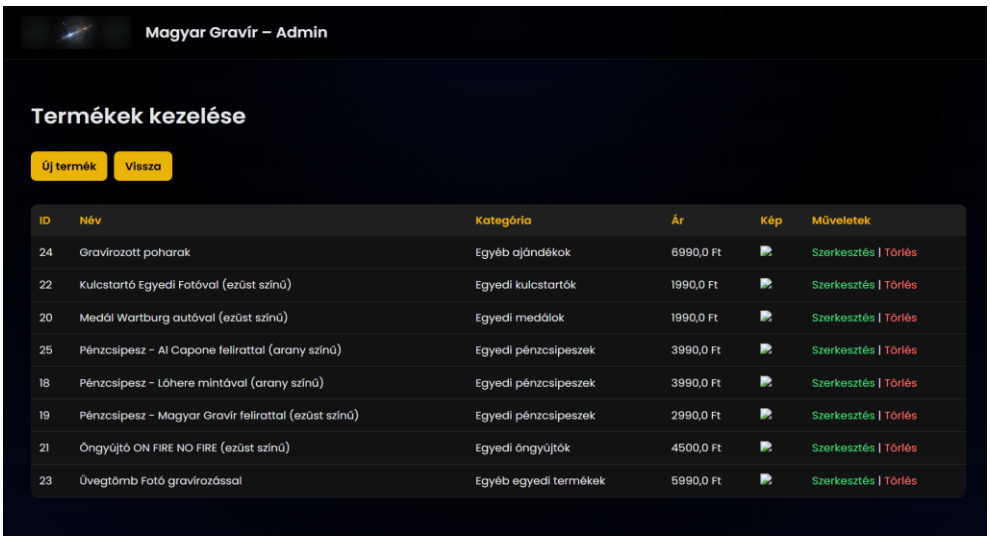
Egyszerű és gyors általános kezelést biztosít a termékek feltöltésére, módosítására vagy törlésére, készletek csökkentésére vagy növelésére. Kategóriák módosítására vagy új kategóriák létrehozására és a legfontosabb, innen kezelhetőek a leadott rendelések is, amely a termékértékesítések lehetőségét adja.

Az admin felület is reszponzív és stílusában megjelenésében a webshop színeit, árnyalatát és háttérét használja a könnyű beazonosíthatóság és egyszerű üzemeltetés segítésére. A jól áttekinthető felület, gyors és egyszerű kezelhetőség fő fontosságú az admin felületnél, a minél hatékonyabb üzemeltetés szempontjából, ezek figyelembevételével készítettük és teszteltük ezt a felületet is.

3.2.1. Termékkezelés

Az admin felületen lehetőség van:

- új termék feltöltésére
- meglévő termék szerkesztésére
- termék törlésére



8. ábra: Admin felület – Termék kezelése fül

Itt megadható a termék főbb adatai, amelyek a webshopban jelennek meg. Új termék hozzáadásánál, vagy szerkesztésénél megadható a termék neve, leírása, ára. Itt választható ki melyik fő kategóriába kerüljön a webshopban, illetve készleten lévő darabszáma és a termék aktív státusza is megadható. A termékekhez kép is feltölthető, amely a szerver uploads mappájába kerül eltárolásra.

3.2.2. Kategóriakezelés

A kategóriák segítségével rendszerezhetőek a webshop termékei.

Az admin felületen a kategóriák:

- létrehozhatóak
- szerkeszthetőek
- törölhetőek
- egyedi státusz megadható

Ezekkel a funkciókkal a webshopban jól átlátható rendszerezhető és a felhasználók számára egyértelmű termék kategorizálás történik.

3.2.3. Rendelések kezelése

Az admin felület listázza a beérkezett rendeléseket, ahol megtekinthető:

- ID (növekményes sorszám, automatikus)
- Név (felhasználói név)
- Email (felhasználói e-mail cím)
- Dátum (rendelés leadásának dátuma)
- Állapot („Feldolgozás alatt”, vagy „Rendelés teljesítve” vagy „Rendelés törölve”)
- Művelet (megnyithatóak további részletek)

A rendelés részleteinek megnyitásával láthatjuk:

- a rendelés sorszámát
- a rendelő nevét, email címét és pontos címét
- az esetleges hozzáfűzött megjegyzést
- a rendelés leadásának dátumát
- a megrendelt termékeket, annak mennyiségét és árát

- a rendelések árának teljes összegét
- és „Rendelés teljesítve” „Rendelés törölve” és „Végleges törlés” gombot

A webshop kezdeti szakaszában innen manuálisan történik a rendeléshez szükséges adatok kinyerése, a rendelés előkészítése és feladása.

4. TESZTELÉS

A fejlesztés során folyamatos tesztelés történt annak érdekében, hogy a rendszer hibamentesen és zavartalanul működhessen, a felhasználói és az admin oldalról is.

A folyamatos tesztelés mellett külön figyelmet fordítottunk a továbbiak teszteltelésére:

- termék hozzáadás
- termék módosítás
- termék törlés
- kategória hozzáadása
- kategória törlése
- regisztráció
- bejelentkezés
- kosár működése
- rendelés leadása

A tesztelések során a rendszer stabil működést mutatott. A megjelenítések reszponzívan működnek többféle eszközön.

A tesztelés további részletei a „Teszt dokumentáció” -ban érhetőek el.

5. ÖSSZEFOGLALÁS

A projekt során egy működőképes webshop rendszer készült, amely lehetővé teszi egyedi gravírozott termékek értékesítését.

A rendszer fő funkciói:

- termékek megjelenítése
- kosár használata
- rendelés leadása
- admin felületen keresztüli termékkezelés

A projekt jól bővíthető, így később további funkciók is hozzáadhatóak, például online fizetési rendszer vagy fejlettebb felhasználói kezelési lehetőségek.

A kor szellemének megfelelően szeretnénk egy AI alapú képszerkesztővel is bővíteni az oldalt, ahol a felhasználók az egyedi elképzeléseiket szöveges, vagy feltöltött kép formájában közvetlenül az AI által elkészített látványterven tekinthetik meg.

6. IRODALOMJEGYZÉK

Könyvek:

Freeman, Eric – Robson, Elisabeth: HTML és CSS – Webdesign alapok, Kiskapu Kiadó, 2019.

Online források:

Microsoft Docs: ASP.NET Core dokumentáció

https://learn.microsoft.com/aspnet/core

SQLite hivatalos dokumentáció

https://www.sqlite.org/docs.html

W3Schools Online Web Tutorials

https://www.w3schools.com

Egyéb források:

Saját fejlesztésű projekt forráskódja (MagyarGravir Webáruház)

Visual Studio fejlesztői környezet dokumentációja